



Sentinel

Real-time fraud detection and prevention in digital lending

A working prototype: rules + machine learning + behavioural signals, with an LLM used only to explain the decision.

· Roll number

FastAPI · React · PostgreSQL / pgvector · Claude (Anthropic API or AWS Bedrock)

Why rule-only fraud systems struggle

Fraud changes faster than rules

Fraud patterns change every few weeks. Writing and releasing a new rule takes much longer than that.

False positives cost real customers

If a genuine customer gets blocked, they simply apply somewhere else. This loss never shows up in a fraud report.

There is very little time

The decision has to happen while the customer is waiting on the screen. A few hundred milliseconds is already too slow.

A score alone is not useful

An analyst cannot do much with "risk = 0.87". They need to know *why* before they can act on it.

So the real requirement is: decide in milliseconds, keep learning without a new release, avoid blocking good customers, and explain every decision.

Four things that shaped the design

- **Behaviour is harder to fake than documents.** A fraudster can buy a stolen PAN, but typing speed, pasting, corrections and how long someone spends on the form are much harder to imitate.
- **Velocity is the strongest signal, and it is cheap.** "How many different applicant names used this device today" catches a fraud ring by its 4th application — but only if you count it at the moment of scoring, not in a nightly job.
- **Rules and ML are better together.** Rules work on day one and explain themselves. The ML model adapts. An anomaly model catches patterns that nobody has labelled yet.
- **The output should be an explanation, not a number.** Evidence, matching fraud type, and what to check next — so the analyst can decide quickly.

One fast decision with four possible outcomes

✓ ALLOW

Application continues normally. Two thirds of the demo traffic.

! STEP UP

Ask for OTP or liveness check. Extra friction, not a rejection.

⌵ REVIEW

Held for a fraud analyst, with the case already prepared.

✗ BLOCK

Blocked and raised as a fraud case.

DECISION TIME (P95)

22 ms

Only rules and models are in this path. The LLM is called afterwards.

MODEL ACCURACY (ROC-AUC)

0.939

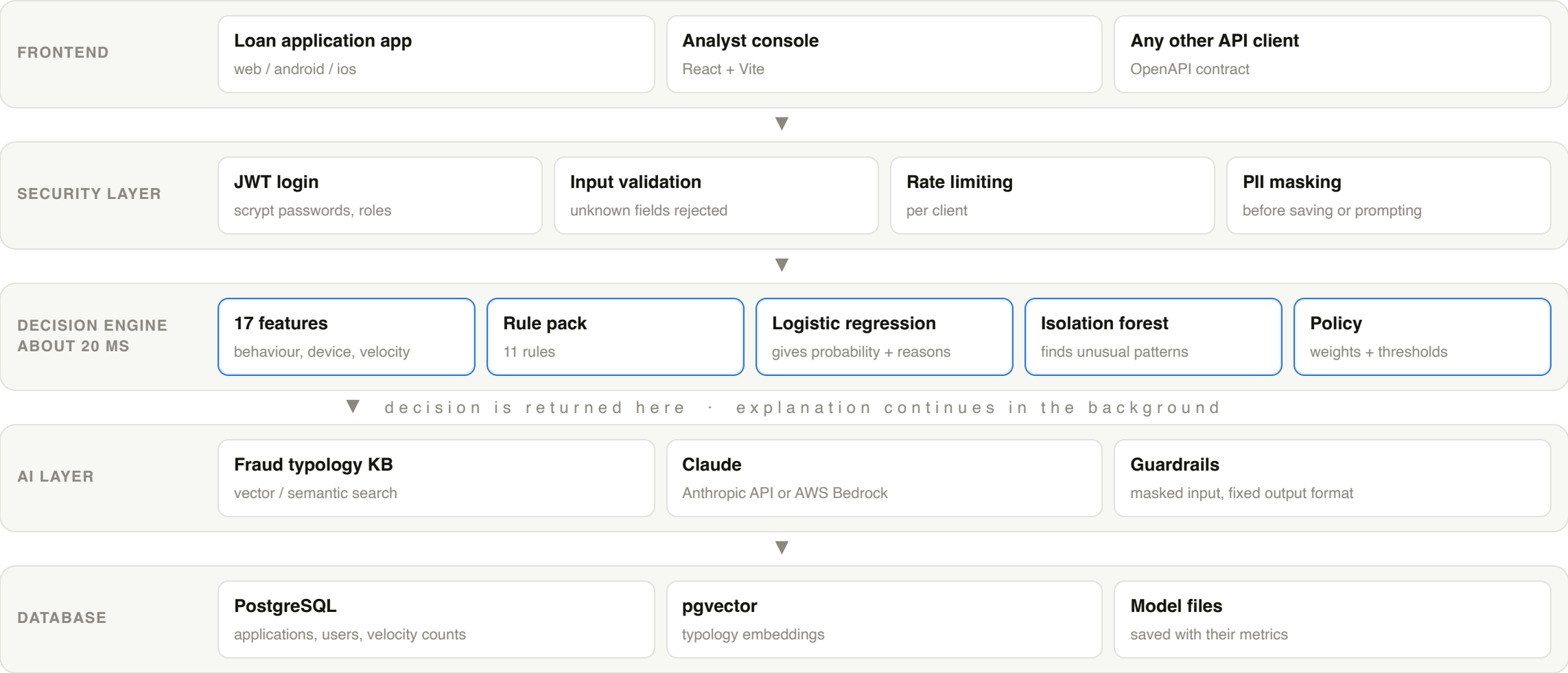
82% of fraud caught at the review threshold, with a 3.9% false-positive rate.

LEARNING LOOP

Instant + retrain

An analyst marking fraud blocks that device immediately, and the case is added to the training data.

Five layers, one decision path



The analyst's verdict goes back into the database: the device is denylisted right away, and the case is used the next time the model is trained.

Simple enough to check by hand

$$\begin{aligned} \text{risk} &= 0.55 \times \text{ML model probability} \\ &+ 0.20 \times \text{anomaly score} \\ &+ 0.25 \times \text{rule score} \end{aligned}$$

BLOCK	risk ≥ 0.85, or a hard-block rule fires
REVIEW	risk ≥ 0.60
STEP UP	risk ≥ 0.35
ALLOW	below that

Weights and thresholds are config, not code

They are read from environment variables, so a risk team can retune them without changing code or redeploying.

Rules combine with noisy-OR

Several weak rules add up to real suspicion, but they can never reach certainty on their own. Only a hard-block rule — for example a device an analyst already confirmed as fraud — forces a BLOCK.

Three models, three jobs

Rules catch what we already know and explain it. The classifier learns from labelled fraud. The isolation forest flags applications that simply do not look like normal traffic.

What the model actually looks at

Behaviour on the form

Time spent · typing speed · paste events · corrections · tab switches · time of day

A real applicant hesitates and corrects mistakes. A script does not. Someone filling their fifth form of the morning types unusually fast.

Device and network

Emulator / rooted device · VPN or proxy · IP location vs declared address

Weak on their own — plenty of genuine users have a VPN — but useful in combination.

Velocity (counted live)

Applications per device in 24h · per IP in 24h · **distinct applicant names per device**

These are counted from the database at scoring time, so there is no batch delay.

Credit and identity

Loan-to-income ratio · loan size · account age · email domain age · prior defaults

Left out on purpose

Name, gender, religion, caste, age group, locality. The model looks at behaviour and transaction facts only.

Six applications from the same device

Each of these applications looks fine on its own: no emulator, no VPN, normal typing speed, an old account, and a reasonable loan amount. A per-application rule engine would pass all six. Sentinel notices the pattern across them.

#	APPLICANT	RISK	DECISION	MAIN REASON
1	Rahul Verma	0.15	✓ ALLOW	IP location differs from address (model, +0.27)
2	Sneha Rao	0.10	✓ ALLOW	Account age (model, +0.26)
3	Imran Qureshi	0.21	✓ ALLOW	IP location differs from address (model, +0.27)
4	Anita Joshi	0.44	! STEP UP	Same device used by 3 different applicant names in 24h
5	Vivek Nanda	0.55	! STEP UP	Same device used by 4 different applicant names in 24h
6	Farah Sheikh	0.59	! STEP UP	Same device used by 5 different applicant names in 24h

It escalates to **extra verification** rather than a block, because six clean-looking applications are suspicious but not proof. Once the analyst confirms one of them as fraud, the **7th application from that device is blocked straight away**, with CONFIRMED_FRAUD_DEVICE as the first reason.

Every decision comes with its reasons

RULE Plain facts

"Same device used by 5 different applicant names in 24h"
"Form filled by paste/automation (9 pastes at 22 chars/s)"
"Email domain registered 4 days ago"

A rule says what happened, not just a number. This is what an auditor would read.

MODEL Real contributions

"Typing speed raises risk (contribution +1.35 log-odds)"

I used logistic regression on purpose. Because it is linear, each feature's contribution is simply $\text{coefficient} \times \text{scaled value}$ — the actual maths of the decision, not an estimate from a separate explainer.

Matching a known fraud type

The case is turned into a vector and matched against a small knowledge base of fraud typologies:

Device farm / application ring similarity 0.81

Suggested control: bind the device fingerprint, block the device across the portfolio, force video KYC for applications sharing it.

So the analyst sees what it looks like *and* what usually stops it.

One list of evidence

Rule hits and model reasons are merged into a single ranked list, so the API response, the console and the saved record all show the same explanation.

The LLM explains the decision, it does not make it

Where it fits

The decision is already made and sent back before Claude is called. It then receives the decision, the rules that fired, the model reasons and the matched typologies, and writes a short summary plus the checks the analyst should do.

Because of this, if the LLM is slow or unavailable, only the wording of the case file changes. The decision itself is never delayed or altered. If there is no API key, a deterministic template is used instead — that fallback is part of the product, not a placeholder.

Guardrails

- PAN, Aadhaar, mobile, email and card numbers are **masked before the prompt is built**
- The system prompt tells it not to invent facts, not to guess masked values, and not to use protected attributes
- **Structured output** — it must return fixed fields and one of four allowed actions
- The output is masked again on the way back
- 20 second timeout and one retry, then fall back — the call can never crash the decision path

Works on Bedrock too

One environment variable switches between the Anthropic API and the same model on AWS Bedrock. Credentials come from the SDK / IAM role, never from the code.

Source is always shown

The console displays which model wrote the summary (anthropic:claude-opus-5 or rule_based_fallback) and says the decision came from the rules and models.

Vector search

Typology embeddings are stored next to the data (a pgvector column in production). Switching to Bedrock Titan embeddings only changes one function.

How the system improves over time

STEP 1 — IMMEDIATE

Analyst confirms fraud

The device and IP are denylisted for the whole portfolio starting from the very next application, using a hard-block rule. No retraining or deployment needed.

STEP 2 — ON DEMAND

Retrain

Every labelled case is added to the training data with **12x the weight** of a synthetic sample. Only an admin can trigger it, it is logged, and the response shows the old and new metrics side by side.

STEP 3 — SAFETY CHECK

Promote only if it is not worse

If the new model loses more than **0.01 ROC-AUC** compared to the current one, it is **not promoted**. The API reports this and the old model keeps running.

In testing with a single labelled case, ROC-AUC moved from 0.9386 to 0.9339 — a drop of 0.005, which is what one sample of noise looks like. That is inside the 0.01 tolerance, so the new model was promoted and the API returned both numbers. A larger drop would have kept the old model serving. I added this check after realising a few bad labels could otherwise quietly make the system worse.

RESULTS

Measured on test data and on the live demo

MODEL QUALITY (1,600 HELD-OUT APPLICATIONS)	VALUE
ROC-AUC	0.939
PR-AUC	0.861
Recall at review threshold	82.2%
Precision at review threshold	68.2%
False-positive rate	3.9%

The training data is synthetic, but deliberately not easy: **40% of the fraud cases only show some of the warning signs**, there is 1.5% wrong labelling, and genuine users share devices, use VPNs and apply at 3 a.m. My first version of the generator gave AUC 1.00, which just meant the fake data was too obvious.

LIVE DEMO RUN (ON POSTGRESQL)	VALUE
Applications scored	60
Decision time p50	29 ms
Decision time p95	37 ms
Allowed without friction	66.7%
Review or blocked	23.3%
Tests passing	20 / 20

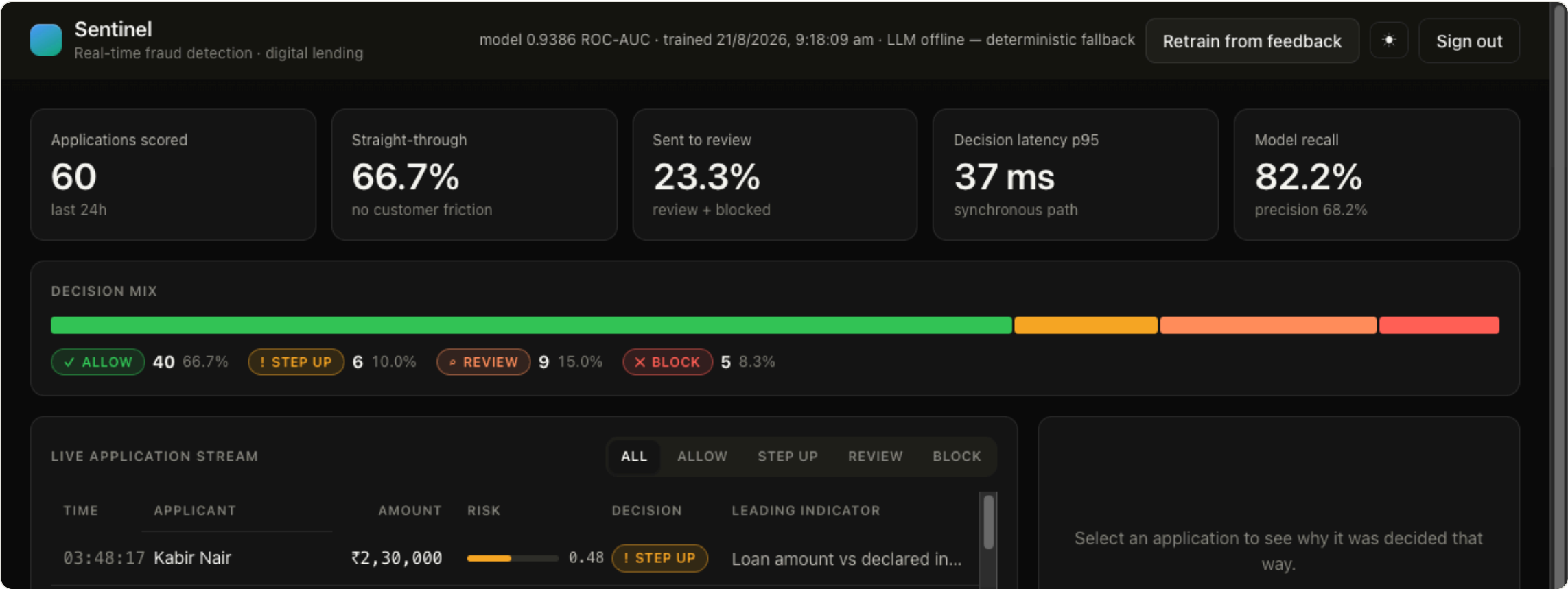
The demo stream deliberately contains about 15% fraud plus a device-farm burst, so the straight-through rate here is much lower than a real portfolio would see. On SQLite the same run decides in 18 ms p50; the extra milliseconds above are the hop to the database container.

About the 68% precision

It means roughly one in three reviewed cases turns out to be genuine. Those cases are only *reviewed*, not blocked. The thresholds are set so that blocking needs strong evidence, while suspicion only adds an extra check.

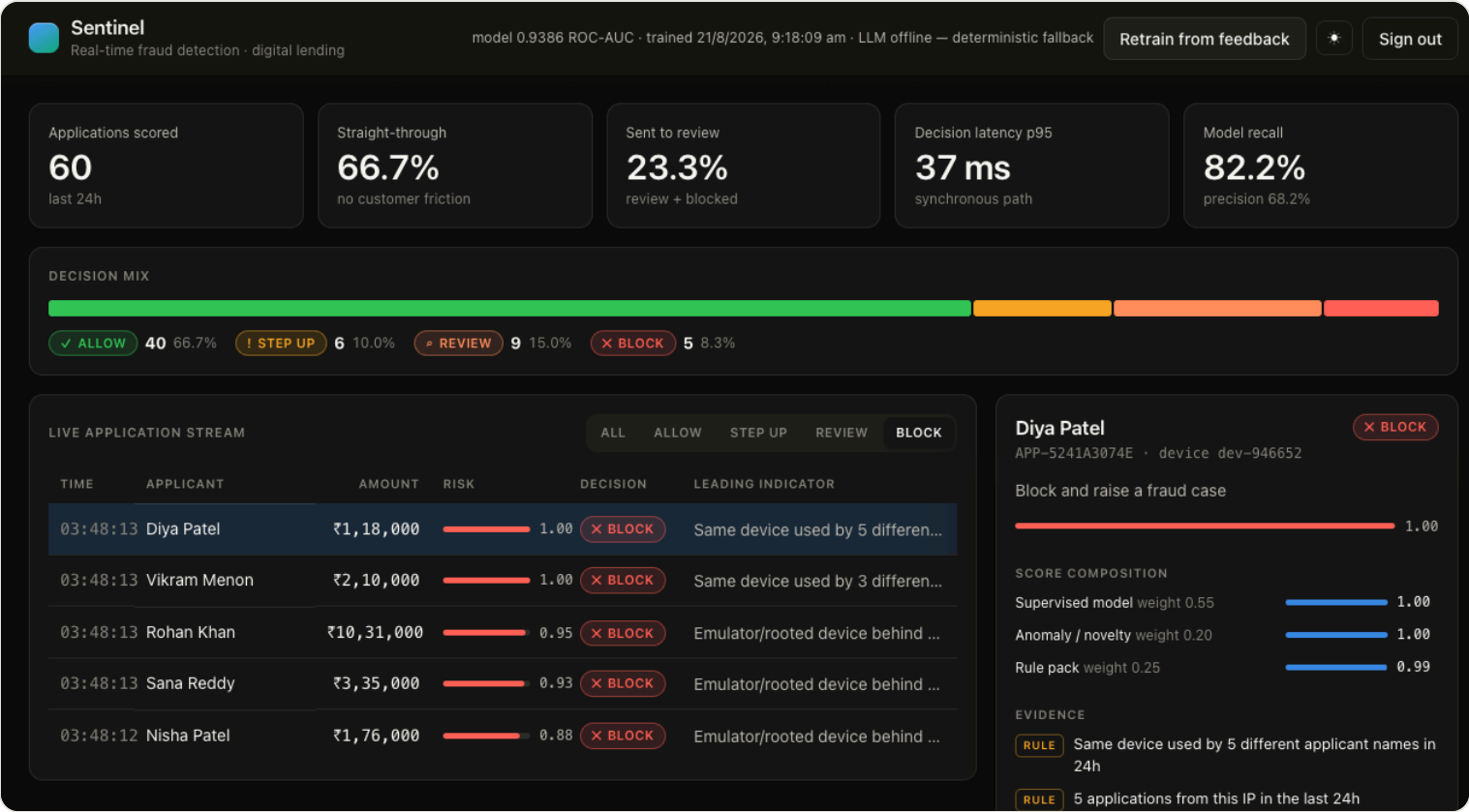
Live application stream

Every decision as it happens, with the main reason next to it. Each decision has an icon and a label, so colour is never the only way to read it.



The case file an analyst sees

Score breakdown, ranked evidence, the matched fraud type with its control, the AI summary with its source, and the two buttons that feed the learning loop.



Handled in the code, not just mentioned

Security

- **Login** JWT tokens (HS256, 60 min), passwords hashed with scrypt and a per-user salt, compared in constant time
- **Roles** analyst vs admin; only an admin can retrain the model, and it is logged
- **Input validation** strict schemas — number ranges, IPv4 check, fixed options, and unknown fields are rejected
- **Rate limiting** per client on login and scoring
- **No hardcoded secrets** everything comes from environment variables; `.env` is git-ignored and `.env.example` is blank. With no JWT secret set, a random one is generated instead of using a default
- **CORS** a fixed list of allowed origins, never *

Responsible AI

- **The LLM never decides** — rules and models decide, it only explains
- **PII is masked** before anything is saved or sent to a model
- **No protected attributes** in the features, and the prompt forbids using them
- **Human in the loop** — suspicion leads to extra verification; only an analyst can confirm fraud
- **Auditable** — features, scores, rules, evidence and the summary are saved per application, so a decision can be explained later
- **Source shown** — the console names the model that wrote each summary

Engineering practices I followed

API first

Every feature is an API endpoint with auto-generated OpenAPI docs at /docs. The React console is just one client; a loan origination system could be another. Nothing is only reachable through the UI.

One file, one job

features · rules · model · knowledge · llm · decision · security · schemas · db · config. The decision is assembled in one place, and the feature order is defined in one place because the model depends on it.

Configuration, not constants

Weights, thresholds, model name, rate limits, database URL and LLM backend all come from the environment. The same code runs on SQLite locally and PostgreSQL + pgvector in the cloud.

20 tests

Decision thresholds, rules and their wording, PII masking, login and roles, and input validation (bad IP, negative amount, unknown field). Two of them test the behaviour that matters most:

- ▶ six applications from one device **must** escalate by the last one
- ▶ after fraud is confirmed, the next clean-looking application from that device **must** be blocked

Easy to run

make setup · make api · make web · make demo · make test

The first start trains the model and loads the knowledge base automatically. No cloud account or API key is needed to try it — the AI layer falls back to its offline path.

Known limitations and how I would fix them

SIMPLIFICATION	WHY IT IS OKAY NOW	WHAT TO DO LATER
Vector search runs in Python	The knowledge base has about 10 documents, so a direct cosine calculation is instant	Move to pgvector ordering once the KB grows
Local TF-IDF embeddings	Works offline, costs nothing, and is deterministic	Replace one function with Bedrock Titan or Voyage embeddings
Rate limiting is in memory	Correct for a single API instance	Use Redis or the API gateway when running more than one
Training data is synthetic	Real lending data is not available to a student project	Use real labelled history — the feedback loop is already built
SQLite by default	Anyone can run the project with no setup	Already supported: set DATABASE_URL, docker-compose file included
Console refreshes every 2s	The real-time part is the 22 ms decision, not the dashboard	Use server-sent events if many analysts use it at once

Next steps I would work on: linking fraud rings by shared bank accounts and addresses (not just devices), checking loan stacking across lenders, and running a new model in shadow mode before promoting it.

What this prototype shows

Against the problem statement

- **Real time** — 22 ms p95, fast enough to run inside the application flow
- **Self-learning** — analyst verdicts block devices instantly and retrain the model, with a safety check before promotion
- **Fewer false positives** — four outcomes, so suspicion costs an OTP instead of a customer
- **New fraud patterns** — an anomaly model that does not need labels
- **Customer trust** — PII masked, no protected attributes, every decision explainable

How to run it

```
cp .env.example .env
make setup
make api    # port 8000, docs at /docs
make web    # port 5173, analyst console
make demo   # simulated traffic with a fraud ring
make test   # 20 tests
```

The recorded demo shows traffic arriving live, a fraud ring escalating application by application, the case file with its evidence, confirming fraud, and the next application from that device getting blocked.

Thank you. Happy to explain any part of the code in more detail.